一. 数据库简介和创建

1. 系统数据库

在安装好 SQL SERVER 后，系统会自动安装 5 个用于维护系统正常运行的系统数据库：

（1）master：记录了 SQL SERVER 实例的所有系统级消息，包括实例范围的元数据（如登录帐号）、端点、链接服务器和系统配置设置。

（2）msdb：供 SQL SERVER 代理服务调度报警和作业以及记录操作员的使用，保存关于调度报警、作业、操作员等信息。（备份还原时）

（3）model：SQL SERVER 实例上创建的所有数据库的模板。

（4）tempdb：临时数据库，用于保存临时对象或中间结果集，为数据库的排列等操作提供一个临时工作空间。（每次启动都会重新创建）

（5）Resource：一个只读数据库，包含了 SQL SERVER 的所有系统对象。（隐藏的数据库）

---

2. 数据库的组成

2.1 数据文件

（1）主要数据文件：扩展名为 .mdf ，每个数据库有且只能有一个。

（2）次要数据文件：扩展名为 .ndf ， 可以没有或有多个。

2.2 日志文件

扩展名为 .ldf ，用于存放恢复数据库的所有日志信息。

2.3 数据的存储分配

（1）数据文件和日志文件的默认存放位置为：`\Programe Files\Microsoft SQL Server\MSSQL.1\MSSQL\Data` 文件夹。

（2）数据的存储分配单位是数据页。一页表是一块 8KB 的连续磁盘空间。

（3）页是存储数据的最小空间分配单位，页的大小决定了数据库表中一行数据的最大大小。

WWW.515148.COM

3. SQL 语句 数据库操作

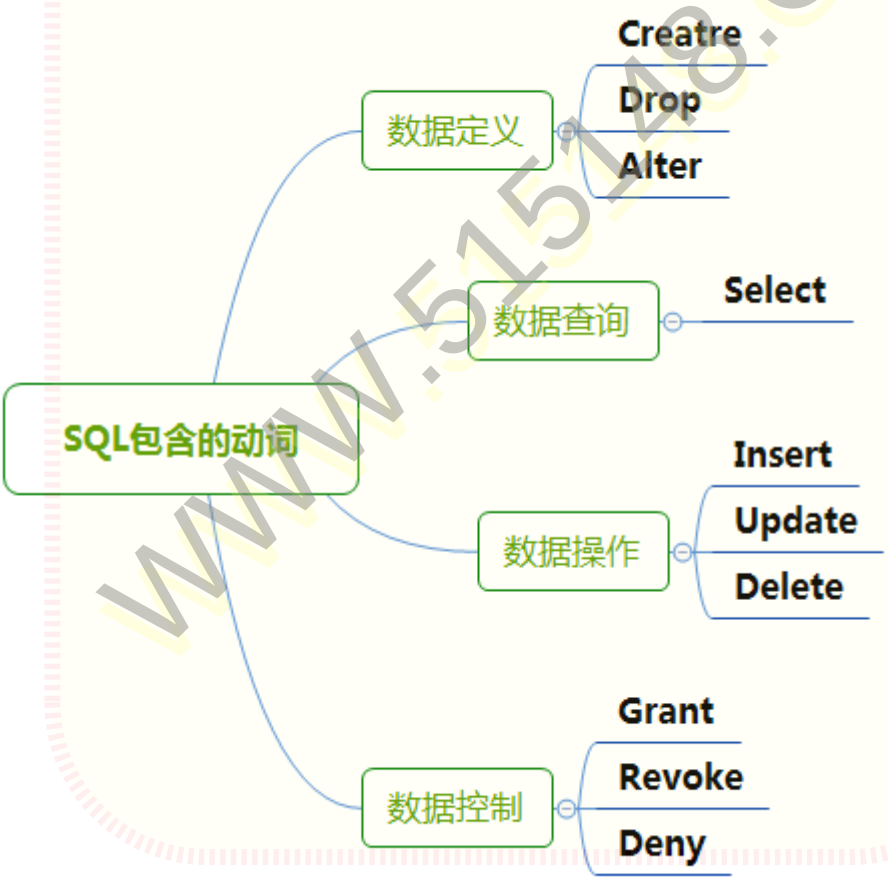
（1）创建数据库

```
CREATE  DATABASE database_name
```

二. SQL 基础

SQL（Structured Query Language，结构化查询语言）是用户操作关系数据库的通用语言。

1. SQL 功能概述



WWW.515148.COM

2. 系统提供的数据类型

2.1 数值数据类型

数据类型	说明	存储空间
bit	bit 数据类型是整型，其值只能是 0、1 或空值。这种数据类型用于存储只有两种可能值的数据，如 Yes 或 No、True 或 False 、On 或 Off.（很省空间的一种数据类型，如果能够满足需求应该尽量多用。）	1 字节
tinyint	tinyint 数据类型能存储从 0 到 255 之间的整数。它在你只打算存储有限数目的数值时很有用。	1 字节
smallint	smallint 数据类型可以存储从- 2 的 15 次幂(-32768)到 2 的 15 次幂(32767)之间的整数。这种数据类型对存储一些常限定在特定范围内的数值型数据非常有用。（如果 tinyint 类型太单调不能满足您的需求，您可以考虑用 smallint 类型，因为这个类型相对也是比较安全的，不接受恶意脚本内容的嵌入。）	2 字节
int	int 数据类型可以存储从- 2 的 31 次幂(-2147483648)到 2 的 31 次幂 (2147483647)之间的整数。存储到数据库的几乎所有数值型的数据都可以用这种数据类型	4 个字节
numeric（p,s） 或 decimal(p,s)	数据类型能用来存储从-10 的 38 次幂-1 到 10 的 38 次幂-1 的固定精度和范围的数值型数据。使用这种数据类型时，必须指定范围和精度。 范围是小数点左右所能存储的数字的总位数。精度是小数点右边存储的数字的位数	最多 17 个字节

2.2 普通编码字符串类型

www.515148.COM

数据类型	说明
char(n)	char 数据类型用来存储指定长度的定长非统一编码型的数据,n 表示字符串的最大长度，取值范围为 1~8000 （若实际字符串控件小于 n,系统自动在后面补空格）
varchar(n)	可变长度的字符串类型,n 表示字符串的最大长度，取值范围为 1~8000。
text	text 数据类型用来存储大量的非统一编码型字符数据。这种数据类型最多可以有 231-1 或 20 亿个字符.

char 和 varchar 的区别：

若某列数据类型为 varchar(20)，存字符串”Jone”时，只占用 4 个字节，而 char（20）会在为填满的空间中填写空格。所以， varchar 类型比 char 类型更节省空间，但它的开销会大一些，处理速度也慢一些。因此，n 值比较小（小于 4），用 char 类型更好些。

2.3 统一编码字符串类型（Unicode）

数据类型	说明
nchar(n)	nchar 数据类型用来存储定长统一编码字符型数据。统一编码用双字节结构来存储每个字符，而不是用单字节(普通文本中的情况)。它允许大量的扩展字符。此数据类型能存储 4000 种字符，使用的字节空间上增加了一倍.
nvarchar(n)	nvarchar 数据类型用作变长的统一编码字符型数据。此数据类型能存储 4000 种字符，使用的字节空间增加了一倍.
ntext	最多可存储 2 的 30 次方-1 将近 10 亿个字符

WWW.515148.COM

三. SQL 数据操作语言

1.数据查询语句

1.1 查询语句的基本结构

```
SELECT <目标列名序列>  --需要哪些列
    From <表名>       --来自哪张表
    [WHERE <行选择条件>]
    [GROUP BY <分组依据列>]
    [HAVING <组>]
    [ORDER BY <排序依据列>]
```

SELECT 子句用于指定输出的字段；
FROM 子句用于指定数据的来源；
WHERE 子句用于指定数据的选择条件；
GROUP BY 子句用于对检索到的记录进行分组；
HAVING 子句用于指定组的选择条件；
ORDER BY 子句用于对查询的结果进行排序；
以上子句中，SELECT 子句和 FROM 子句是必需的，其它是可选的。

WWW.515148.COM

1.2 单表查询

1.2.1 选择表中若干列

（1）查询指定的列

```
SELECT 列名 FROM 表名
```

- 1

例子 ： `SELECT Sname,Sno FROM Student`

（2）查询全部列

```
SELECT * FROM 表名
```

- 1

例子 ： `SELECT * FROM Student`

（3）查询经过计算的列

```
SELECT 列名 FROM 表名
```

- 1

例子 ： `SELECT Sname,year(getdata()) - year(Birthdate) FROM Student`

1.2.2 选择表中的若干元祖

（1）消除取值相同的行：**DISTINCT**

```
SELECT DISTINCT Sno FROM 表名
```

- 1

例子 ： `SELECT DISTINCT Sno FROM Student`

（2）查询满足条件的元祖

查询条件	谓 词
比较	=、>、>=、<=、<、<>、!=、!>、!<
确定范围	BETWEEN…AND、 NOT BETWEEN…AND
确定集合	IN 、NOT IN

WWW.515148.COM

查询条件	谓 词
字符匹配	LIKE 、NOT LIKE
空值	IS NULL、IS NOT NULL
多重条件（逻辑谓词）	AND、OR

a.比较大小

例子 ： `SELECT Sname FROM Student WHERE year(getdata()) - year(Birthdate) < 20`

b.确定范围

BETWEEN…AND 和 NOT BETWEEN…AND 可用于查找属性值在或不在指定范围。

```
列名 | 表达式 | [NOT] BETWEEN 下限值 AND 上限值
```

- 1

BETWEEN…AND 代表的范围是在上限值和下限值之间（**包括**边界值），即为 true。

NOT BETWEEN…AND 代表的范围是**不在**上限值和下限值之间（**不包括**边界值），即为 true。（若判断值为边界值时，为 false）

例子 ： `SELECT Sno,Cno FROM SC WHERE Grade BETWEEN 80 AND 90`

此查询等价于：`SELECT Sno,Cno FROM SC WHERE Grade >= 80 AND Grade <= 90`

例子 ： `SELECT Sno,Cno FROM SC WHERE Grade NOT BETWEEN 80 AND 90`

此查询等价于：`SELECT Sno,Cno FROM SC WHERE Grade < 80 OR Grade > 90`

---

c. 确定集合

IN 运算符的含义：当列中的值和集合中的**某个**常量值相等时，结果为 **True**。

NOT IN 运算符的含义：当列中的值和集合中的**全部**常量值**都不**相等时，结果为 **True**。

WWW.515148.COM

例子 ： `SELECT Sno FROM Student WHERE Dept IN ('信息管理系','计算机系')`
此查询等价于：`SELECT Sno FROM Student WHERE Dept = '信息管理系' OR Dept = '计算机系')`

例子 ： `SELECT Sno FROM Student WHERE Dept NOT IN ('信息管理系','计算机系')`
此查询等价于：`SELECT Sno FROM Student WHERE Dept != '信息管理系' AND Dept != '计算机系')`

---

d. 字符串匹配

Like 运算符用于查找指定列中与匹配串匹配的元祖。

```
列名 [NOT] LIKE <匹配串>
```

通配符	含义
_（下划线）	匹配任意一个字符
%（百分号）	匹配 0 个或多个字符
[]	匹配[]中的任意一个字符。如[abcd]表示匹配 abcd 其中任何一个，若是连续的，可以用 - 表
[^]	不匹配[]中的任意一个字符。如[^abcd]表示不匹配 abcd 其中任何一个，若是连续的，可以用

例子 ：

```
(查询姓“张”的学生详细信息)

SELECT * FROM Student WHERE Sname LIKE '张%'



(查询不姓“张”的学生详细信息)
```

WWW.515148.COM

```
SELECT * FROM Student WHERE Sname NOT LIKE '张%'
```

（查询姓“张”、“李”的学生详细信息）

```
SELECT * FROM Student WHERE Sname LIKE '[张李]%'
```

（查询名字的第二个字为“小” 或 “大”的学生详细信息）

```
SELECT * FROM Student WHERE Sname LIKE '_[小大]%'
```

e. 涉及空值的查询

空值（NULL）在数据库中有特殊含义，表示当前不确定或未知的值。判断是否为 NULL 时，不可用普通的比较运算符，需用 IS NULL

例子 ：`SELECT Sno FROM Student WHERE Grade IS NULL`

1.2.3 对查询结果进行排序

将查询结果按照指定的顺序显示。**ASC** 表示按列值**升序**排列（从上往下，值从大到小）。**DESC** 表示按列值**降序**排列（从上往下，值从小到大）。**默认为 ASC**。

```
ORDER BY <列名> [ASC|DESC]
```

例子 ：`SELECT Sno,Grade FROM SC ORDER BY Grade DESC`

1.2.4 使用聚合函数统计数据

聚合函数也称为统计函数或集合函数，作用是对一组值进行计算并返回一个统计结果。

聚合函数	含义
COUNT(*)	统计表中元祖的个数

www.515148.COM

聚合函数	含义
COUNT([DISTINCT]<列名>)	统计本列的非空列值个数
SUM(<列名>)	计算列值的和值（必须是数值型列）
AVG(<列名>)	计算列值的平均值（必须是数值型列）
MAX(<列名>)	计算列值的最大值
MIN(<列名>)	计算列值的最小值

上述函数除 COUNT(*) 外，其它函数在计算过程中均忽略 NULL 值

```
（统计学生总人数）

SELECT COUNT(*) FROM Student



（统计“001”学号学生的考试平均成绩）

SELECT AVG(Grade) FROM SC WHERE Sno = '001'



（查询“C001”号课程考试成绩的最高分和最低分）

SELECT MAX(Grade) 最高分,MIN(Grade) 最低分 FROM SC WHERE Cno = 'C001'
```

聚合函数不能出现在 **WHERE** 子句中！

1.2.5 对数据进行分组统计

WWW.515148.COM

需要先对数据进行分组，然后再对每个组进行统计。分组子句 GROUP BY。在一个查询语句中，可以用多个列进行分组。
分组子句跟在 WHERE 子句的后面：

```
GROUP BY <分组依据列>[,...n]

    [HAVING <组筛选条件>]
```

（1）使用 GROUP BY 子句

```
（统计每门课程的选课人数，列出课程号和选课人数）

SELECT Cno as 课程号, COUNT(Sno) as 选课人数 From SC Group BY Cno



（统计每个学生的选课门数和平均成绩）

SELECT Sno 学号, COUNT(*)  选课门数,AVG(Grade) 平均成绩 From SC Group BY S
no



带 WHERE 子句的分组（统计每个系的女生人数）

SELECT Dept, COUNT(*)  女生人数 From Student Where Sex = '女' Group BY Dep
t
```

（2）使用 HAVING 子句

HAVING 子句用于对分组后的统计结果再进行筛选，它的功能与 WHERE 子句类似，它用于组而不是单个记录。在 HAVING 子句中可以使用聚合函数，但在 WHERE 子句中不能，通常与 GROUP 子句一起使用。

```
（查询选课门数超过 3 门的学生的学号和选课门数）

SELECT Sno 学号, COUNT(*)  选课门数,AVG(Grade) 平均成绩 From SC Group BY S
no HAVING COUNT(*) > 3
```

（3）WHERE 、GROUP BY 、HAVING 的作用及执行顺序

- **WHERE** 子句用于筛选 **FROM** 子句中指定的数据所产生的行数据。

WWW.515148.COM

- **GROUP BY** 子句用于对经 **WHERE** 子句筛选后的结果数据进行分组。
- **HAVING** 子句用于对**分组后**的统计结果再进行筛选。

可以分组操作之前应用的筛选条件，在 WHERE 子句中指定它们更有效，这样可以减少参与分组的数据行。在 HAVING 子句中指定的筛选条件应该是那些必须在执行分组操作之后应用的筛选条件。

（查询计算机系和信息管理系每个系的学生人数）

第一种：

```
SELECT Dept,COUNT(*) FROM Student GROUP BY Dept Having Dept in('计算机系','信息管理系')
```

第二种：

```
SELECT Dept,COUNT(*) FROM Student WHERE Dept in ('计算机系','信息管理系')GROUP BY Dept
```

以上例子比较：第一种是按照系分组好了之后，只采取所有系中的两个系，显然效率不高。而第二种是先进行 WHERE 筛选条件之后，再进行 GROUP BY 计算，显示更好。

1.3 多表连接查询

若一个查询同时涉及到两张或以上的表，则称为连接查询。

1.3.1 内连接

使用内连接时，如果两个表的相关字段满足条件，则从两个表中提取数据组成新的记录。

```
FROM 表1 [INNER] JOIN 表2 ON <连接条件>
```

- 1

注意：连接条件中的连接字段必须是可比的，必须是语义相同的列。

（查询学生及选课的详细信息）

```
SELECT * FROM Student INNER JOIN SC ON Student.Sno = SC.Sno
```

WWW.515148.COM

（查询计算机系学生的选课情况，列出该学生的名字、所修课程号、成绩）------行选择条件

```
SELECT Sname,Cno,Grade FROM Student INNER JOIN SC ON Student.Sno = SC.Sno WHERE Dept = '计算机系'
```

（统计每个系的平均成绩） ------分组的多表查询

```
SELECT Dept,AVG(Grade) AS AverageGrade FROM Student S INNER JOIN SC ON S.Sno = SC.Sno Group BY Dept
```

（统计计算机系每个学生的选课门数、平均成绩、最高成绩、最低成绩）------分组和行选择条件的多表连接查询

```
SELECT Sno,COUNT(*),AVG(Grade),MAX(Grade),MIN(Grade) FROM Student S JOIN SC ON S.Sno = SC.Sno WHERE Dept = '计算机系' Group BY Dept
```

1.3.2 自连接

自连接是一种特殊的内连接，相互连接的表在物理上是一张表，但在逻辑上可以看做是两张表。

```
FROM 表1 AS T1 JOIN 表1 AS T2
```

- 1

通过为表取别名的方法，可以让物理上的一张表在逻辑上成为两张表。（一定要为表取别名！）

（查询与刘晨在同一个系学习的学生的姓名、所在系）

```
SELECT S1.Sname,S1.Dept FROM Student S1 JOIN Student S2

ON S1.Dept = S2.Dept   ---同一个系的学生
```

WWW.515148.COM

```
WHERE S2.Sname = '刘晨'  ---S2 表作为查询条件

AND S1.Sname != '刘晨'   ----S1 表作为结果表，并从中去掉‘刘晨’本人信息
```

1.3.3 外连接

在**内连接**操作中，只有满足条件的元祖才能出现在查询结果集中。
外连接是只限制一张表中的数据必须**满足**条件，而另一张表的数据可以**不满足**条件。

```
FROM 表 1 LEFT|RIGHT [OUTER] JOIN 表 2 ON <连接条件>
```

- 1

LEFT [OUTER] JOIN 称为**左外连接**，含义是限制**表 2** 中的数据必须满足条件，但不管**表 1** 中的数据是否满足条件，均输出表 1 中的数据。
LEFT [OUTER] JOIN 称为**右外连接**，含义是限制**表 1** 中的数据必须满足条件，但不管**表 2** 中的数据是否满足条件，均输出**表 2** 中的数据。

内连接与外连接的区别：



内连接：表 A 与表 B 进行内连接，则结果为两个表中满足条件的记录集，即 C 部分。
外连接：如果表 A 和表 B 进行左外连接，则结果为 记录集 A + 记录集 C；如果表 A 和表 B 进行右外连接，则结果为 记录集 B + 记录集 C。

```
（查询没有人选的选修课程名）
```

www.515148.COM

```
SELECT Cname FROM Course C LEFT JOIN  SC ON C.Cno = SC.Cno WHERE SC.Cno I
S NULL
```

例子解析：如果存在部分课程为被人选择，则必定在 Course 表中有但在 SC 表中没有出现，即在进行外连接时没人选的课程在与 SC 表**构成的连接结果集中**，对应的 Sno、Cno、Grade 列必定为空，所以只需**在连接后的结果中选出**SC 表中 Sno 或 Cno 为空的元祖即可。

```
（统计计算机系每个学生的选课门数，包括没选课的学生）

SELECT S.Sno AS 学号,COUNT(SC.Cno) AS 选课门数 FROM Student S LEFT JOIN SC
 ON S.Sno = SC.Sno WHERE Dept = '计算机系' GROUP BY S.Sno
```

例子解析： 上述例子要求统计每个学生的….，所以在 GROUP BY 分组时，是按照学生表中的学号来分。而对于聚合函数 COUNT，上述要求统计每个学生的选课门数，若写成 COUNT(S.Sno)或 COUNT(***)，则对没选课的学生都返回 1，**因为在外连接结果中，**S.Sno** 不会是 **NULL**，而 **COUNT(*)函数本身也不考虑 **NULL**，它是直接对元祖个数进行计数。**

注意：在对外连接的结果进行分组、统计等操作时，一定要注意分组依据列和统计列的选择。

1.4 使用 TOP 限制结果集行数

在使用 SELECT 语句进行查询时，有时只需要前几行数据。

```
TOP (expression) [PERCENT] [WITH TIES]
```

- 1

- **expression**：指定返回行数的数值表达式。如果指定了 PERCENT，expression 将隐式转换成 float，否则是 bigint
- **PERCENT**：指定只返回结果集中前 expression% 行数据。
- **WITH TIES**：指定从基本结果集中返回额外的数据行（只有在 SELECT 子句中包含了 ORDER BY 子句时，才能使用）。

TOP 谓词写在 SELECT 单词的后面（如果有 DISTINCT，则在 DISTINCT 后面）。

```
（查询考试成绩最高的 3 个成绩。列出学号、课程号、成绩）
```

WWW.515148.COM

```
SELECT TOP 3 Sno,Cno,Grade FROM SC ORDER BY Grade DESC
```

若要包括并列第 3 名的成绩：

```
SELECT TOP 3 Sno,Cno,Grade WITH TIES FROM SC ORDER BY Grade DESC
```

2.数据更改功能

2.1 插入数据

```
INSERT INTO 表名[(列名)] VALUES (值)
```

- 1

（1）简单插入语句

```
INSERT INTO Student VALUES ('001','陈东','男','1996/6/23','信息管理系')
```

- 1

（2）多行插入语句

```
INSERT INTO SC VALUES('001','C001',90),
                    ('001','C002',30),
                    ('001','C005',NULL)
```

（3）不按表顺序插入语句

按与表列顺序不同的顺序插入数据

```
INSERT INTO Student(Sno,Sname,Sex,Dept) VALUES ('001','陈东','男','1996/
6/23','信息管理系')
```

2.2 更新数据

```
UPDATE 表名 SET 列名 = 值
```

（1）无条件更新

```
UPDATE SC SET Grade = Grade+10
```

WWW.515148.COM

（2）有条件更新

```
（将“C001”号课程的学分改成 5 分）

UPDATE Course SET Grade = 5 WHERE Cno = 'C001'



（将计算机系全体学生的成绩加 5 分）

UPDATE SC SET Grade = Grade+5 FROM SC JOIN Student S ON S.Sno = SC.Sno WH
ERE Dept = '计算机系'
```

2.3 删除数据

```
DELETE [TOP (expression) [PERCENT]]

    FROM 表名
```

（1）无条件删除

```
DELETE FROM Student
```

（2）有条件删除

```
（删除所有考试成绩不合格的学生的选课记录）

DELETE FROM SC WHERE Grade < 60



（删除 Student 表中 2.5%的行数据）

DELETE TOP (2.5) PERCENT FROM Student
```

四. 高级查询

WWW.515148.COM

1. CASE 函数

CASE 函数是一种多分支函数，它可以根据条件列表的值返回多个可能的结果表达式中的一个。

1.1 简单 CASE 函数

```
CASE input_expression

    WHEN when_expression THEN result_expression

    [...n]

    [ELSE else_expression]

END
```

- **input_expression**：所计算的表达式，可以是一个变量名、字段名、函数或子查询。
- **when_expression** ：要与 input _expression 进行比较的简单表达式。简单表达式中不可包含比较运算法，只需给出被比较的表达式或值。
- **else_expression** ： 比较结果均不为 TRUE 时返回的表达式。

```
（查询选了 JAVA 课程的学生的学号、姓名、所在系、成绩，

若所在系为“计算机系”，则显示“CS”；若所在系为“信息管理系”，则显示“IM”；若所在系为“通信工程系”，则显示“COM”）

SELECT S.Sno 学号,Sname 姓名,

    CASE Dept

        WHEN '计算机系' THEN 'CS'

        WHEN '信息管理系' THEN 'IM'

        WHEN '通信工程系' THEN 'COM'

    END AS 所在系,Grade 成绩
```

WWW.515148.COM

```
    FROM Student S JOIN SC ON S.Sno = SC.Sno

    JOIN Course C ON C.Cno = SC.Cno

    WHERE Cname = 'Java'
```

1.2 搜索 CASE 函数

简单 CASE 函数只能将 **input_expression** 与一个单值进行比较，如果需要跟一个范围内的值进行比较，就需要**搜索 CASE 函数**。

```
CASE

    WHEN Boolean_expression THEN result_expression

    [...n]

    [ELSE else_expression]

END
```

- **Boolean_expression**：比较表达式，可以包含比较运算符，直接将两者进行比较。

上述例子也可以用搜索 CASE 函数：

```
SELECT S.Sno 学号,Sname 姓名,

    CASE

        WHEN Dept = '计算机系' THEN 'CS'

        WHEN Dept = '信息管理系' THEN 'IM'

        WHEN Dept = '通信工程系' THEN 'COM'
```

WWW.515148.COM

```
    END AS 所在系,Grade 成绩
    FROM Student S JOIN SC ON S.Sno = SC.Sno
    JOIN Course C ON C.Cno = SC.Cno
    WHERE Cname = 'Java'
```

（查询 C001 课程的考试情况，列出学号和成绩，然后根据成绩划分等级）

```
SELECT S.Sno 学号,Sname 姓名,
    CASE
        WHEN Grade >= 90 THEN '优'
        WHEN Grade BETWEEN 80 AND 99 THEN '良'
        WHEN Grade BETWEEN 70 AND 79 THEN '中'
        WHEN Grade BETWEEN 60 AND 69 THEN '及格'
    END AS 成绩
    FROM  SC ON WHERE Cno = 'C001'
```

2. 子查询

如果一个 SELECT 语句嵌套在另一个 SELECT、INSERT、UPDATE 或 DELETE 语句中，则称为子查询或内层查询；而包含子查询的语句称为主查询。

子查询通常用于满足下列需求之一：

- 把一个查询分解成一系列的逻辑步骤
- 提供一个列表作为 WHERE 子句和 IN、EXISTS、ANY、ALL 的目标对象
- 提供由外层查询中每一条记录驱动的查询

子查询通常有几种形式：

WWW.515148.COM

- WHERE 列名 [NOT] IN (子查询)
- WHERE 列名 比较运算符 (子查询)
- WHERE EXISTS(子查询)

2.1 使用基于集合测试的嵌套子查询

使用嵌套子查询进行基于集合的测试时，子查询返回的是一个值列表，外层查询通过运算符 IN 或 NOT IN，对子查询返回的结果集进行比较。

```
SELECT <查询列表> FROM ...

WHERE <列名> [NOT] IN (

SELECT <列名> FROM)
```

包含这种子查询形式的查询语句是分步骤实现的，即先执行子查询，然后在子查询的结果基础上执行外层查询（**先内后外**）。子查询返回的结果是一个集合，外层查询就是在这个集合上使用 IN 运算符进行比较。

```
（查询与刘晨在同一系的学生）

SELECT Sno,Sname,Dept FROM Student WHERE Dept IN

    (SELECT Dept From Student Where Sname = '刘晨')



（查询选修了 JAVA 课程学生的姓名）

SELECT Sname FROM Stduent WHERE Sno IN(

    SELECT Sno FROM SC WHERE Cno IN(

        SELECT Cno FROM Course WHERE Cname = 'JAVA'))



第二个例子也可以用了连接查询来做：
```

WWW.515148.COM

```
SELECT Sname From Student S JOIN SC ON S.Sno = SC.Sno JOIN Course C ON C.
Cno = SC.Cno WHERE Cname = 'JAVA'
```

上述的例子可以看出**子查询**和**连接查询**可以互通，但在某些情况下是不可以的：

```
（统计选了 JAVA 课程的学生的姓名和所在系）

子查询：

SELECT Sno 学号,COUNT(*) 选课门数,AVG(Grade) 平均成绩

    FROM SC WHERE Sno IN(

      SELECT Sno FROM SC JOIN Course C

        ON C.Cno = SC.Cno

          WHERE Cname = 'JAVA')

     GROUP BY Sno
```

这个查询不能完全用**连接查询** 实现，**因为这个查询的语义是要先找出选了 JAVA 课程的学生，然后再计算这些学生的选课门数和平均成绩。**

连接查询：

```
SELECT Sno 学号,COUNT(*) 选课门数,AVG(Grade) 平均成绩

       FROM SC JOIN Course C ON C.Cno = SC.Cno WHERE Cname = 'JAVA'
 GROUP BY Sno
```

以上查询是错误的，查出来学生的选课门数都是 1 ！！！ 实际上这个 1 指的是 JAVA 这一门课程，，其平均成绩也是 JAVA 成绩。

【注意：】连接查询和子查询的区别：★★★★★

- 之所以这样，是因为在执行有**连接操作**的查询时，系统首先将所有被连接的表**连接成一张大表**，这张大表中的数据全部**满足连接条件**的数据。之后再在这张连接后的大表上执行 WHERE 子句，然后是 GROUP BY 子句。

WWW.515148.COM

执行完 WHERE 子句之后，连接的大表中的数据就只剩下 JAVA 这一门课程的情况了，显然不符情况。

- 对于含有**嵌套的子查询**的查询，是先执行子查询，然后在子查询的结果基础上再执行外层查询。

【注意：】在子查询中否定和在外查询中否定的区别 ★★★★★

IN 和 != 的搭配 相较于 NOT IN 和 =的搭配是否相同？
在子查询中否定和在外查询中否定的区别？

（查询没选 C001 课程的学生的姓名和所在系）

1.多表连接：

```
SELECT Sname,Dept FROM Student S JOIN SC ON S.Sno= SC。Sno WHERE Cno != '
C001'
```

2. 嵌套子查询：

1）在子查询中否定

```
SELECT Sname,Dept FROM Student
    WHERE Sno NOT IN(
      SELECT Sno FROM SC WHERE Cno = 'C001')
```

2）在外层查询中否定

```
SELECT Sname,Dept FROM Student
    WHERE Sno IN(
```

WWW.515148.COM

```
    SELECT Sno FROM SC WHERE Cno != 'C001')
```

这个例子，连接查询是错误的，嵌套子查询中方法一在子查询中的否定是错误的！**嵌套子查询中方法二在外查询中的否定是正确的！**

【连接查询】：

- 上面已经讲过，对于**连接查询**，所有的条件都是在连接之后的结果表上进行的，而且是逐行进行判断。一旦发现满足要求“Cno != ‘C001’”，则此行满足条件。所以最后的结果既包括没有选 C001 课程的学生，也包含选了 C001 同时选了别的课程的学生。

【含有嵌套的子查询】：

- 对于含有**嵌套的子查询**的查询，是先执行子查询，然后在子查询的结果基础上再执行外层查询。而且在子查询中也是逐行判断的，当发现有满足条件的数据时，将此行数据作为外行查询的一个比较条件。

本例要查询的是某个学生所选的全部课程中均不包含 C001 课程，如果将否定放在子查询中，则查出的学生既包括没有选 C001 课程的学生，也包含选了 C001 同时选了别的课程的学生。显然，这个否定的范围不够。

通常情况下，对于这种带有部分否定条件的查询都应该用子查询来实现，而且应该放在外层！

2.2 使用比较测试的嵌套子查询

```
SELECT <查询列表> FROM...

    WHERE <列名> 比较运算符 (

      SELECT <列名> FROM ...)
```

使用嵌套子查询进行比较测试时，**要求子查询只能返回单个值**。外层查询一般通过比较运算符（=、<>、 <= 、>=），将外层查询中某个列的值与子查询返回的值进行比较。

```
(查询选了 C004 课程且成绩高于此课程的平均成绩的学生的学号和成绩)

SELECT Sno, Grade FROM SC
```

WWW.515148.COM

```
    WHERE Cno = 'C004' AND Grade >(

      SELECT AVG(Grade) FROM SC WHERE Cno = 'C004'
```

2.2 使用 SOME 和 ALL 嵌套子查询

当子查询返回单值时，可以使用**比较运算符**进行比较，但返回**多值**时，就需要通过 **SOME 和 ALL** 修饰，同时必须使用比较操作符！

```
WHERE <列名> 比较运算符 [SOME | ALL](子查询)
```

- 1

（查询其他学期开设的课程中比第一学期开设课程的学分少的课程名、开课学期、学分）

```
SELECT Cname,Semester,Credit FROM Course

    WHERE Credit < SOME(

      SELECT Credit FROM Course

        WHERE Semester = 1)

AND Semester != 1
```

该语句实际上等价于：查询比第一学期学分最高的课程的学分小的其它学期的课程名、开课学期和学分：

```
SELECT Cname,Semester,Credit FROM Course

    WHERE Credit < (

      SELECT MAX(Credit) FROM Course

        WHERE Semester = 1)

AND Semester != 1
```

WWW.515148.COM

（查询至少有一次成绩大于或等于 90 分的学生的学号及选修的全部课程的课程号和考试成绩）

```
SELECT Sname,Cno,Grade FROM Student S

    JOIN SC ON S.Sno = SC.Sno

     WHERE S.Sno = SOME (

      SELECT Sno FROM SC

         WHERE Grade >= 90)
```

该语句实际上是查询成绩大于或等于 90 的学生的学号及选修的全部课程的课程号和考试成绩
，可用子查询来做：

```
SELECT Sname,Cno,Grade FROM Student S

    JOIN SC ON S.Sno = SC.Sno

     WHERE S.Sno IN (

      SELECT Sno FROM SC

         WHERE Grade >= 90)
```

WWW.515148.COM

字段类型	描述
bit	0或1的整型数字
int	从-2^31(-2,147,483,648)到2^31(2,147,483,647)的整型数字
smallint	从-2^15(-32,768)到2^15(32,767)的整型数字
tinyint	从0到255的整型数字
decimal	从-10^38到10^38-1的定精度与有效位数的数字
numeric	decimal的同义词
money	从-2^63(-922,337,203,685,477.5808)到2^63-1(922,337,203,685,477.5807)的货币数据，最小货币单位千分之十
smallmoney	从-214,748.3648到214,748.3647的货币数据，最小货币单位千分之十
float	从-1.79E+308到1.79E+308可变精度的数字
real	从-3.04E+38到3.04E+38可变精度的数字
datetime	从1753年1月1日到9999年12日31的日期和时间数据，最小时间单位为百分之三秒或3.33毫秒
smalldatetime	从1900年1月1日到2079年6月6日的日期和时间数据，最小时间单位为分钟
timestamp	时间戳，一个数据库宽度的唯一数字
uniqueidentifier	全球唯一标识符GUID
char	定长非Unicode的字符型数据，最大长度为8000
varchar	变长非Unicode的字符型数据，最大长度为8000
text	变长非Unicode的字符型数据，最大长度为2^31-1(2G)
nchar	定长Unicode的字符型数据，最大长度为8000
nvarchar	变长Unicode的字符型数据，最大长度为8000
ntext	变长Unicode的字符型数据，最大长度为2^31-1(2G)
binary	定长二进制数据，最大长度为8000
varbinary	变长二进制数据，最大长度为8000
image	变长二进制数据，最大长度为2^31-1(2G)

WWW.515148.COM